

Day 5: Sequence Models & Transformers

- **Sequence Models & Transformers** — From recurrence to attention
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Sequence Modeling
- Recurrent Neural Networks
- Sequence to Sequence
- Building Intuition for Attention
- Standard Notation and Why Transformers Won

1 · Sequence Modeling

1.1 Why Sequences are Different

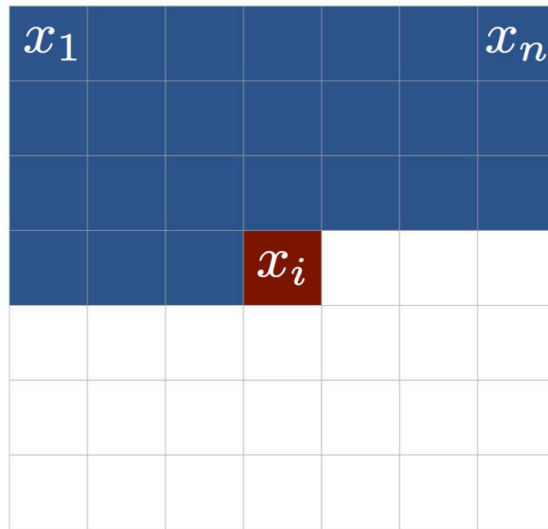
- Inputs/outputs have **variable length**: text, audio, time series
- Order matters: 'dog bites man' $\neq$ 'man bites dog'
- Long-range dependencies span many steps
- Need parameter sharing **across positions in time**
- Two ideas today: recurrence, then attention

1.2 Autoregressive Factorization

- Any joint factorizes by the chain rule of probability:
- $p(x) = \prod_{i=1}^n p(x_i \mid x_1, \dots, x_{i-1})$
- Model each conditional with a shared network
- Train by next-token prediction (max likelihood)
- Generate by sampling one token at a time

1.2 Autoregressive Factorization

Images as sequences: PixelRNN



$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1})$$

Van den Oord et al. (2016)



1.3 Sharing Across Time

- Reuse the **same** weights at every time step
- Like CNNs share across space, RNNs share across time
- Handles arbitrary length with a fixed parameter count
- A hidden state carries a summary of the past
- Output head: softmax over the vocabulary

2 · Recurrent Neural Networks

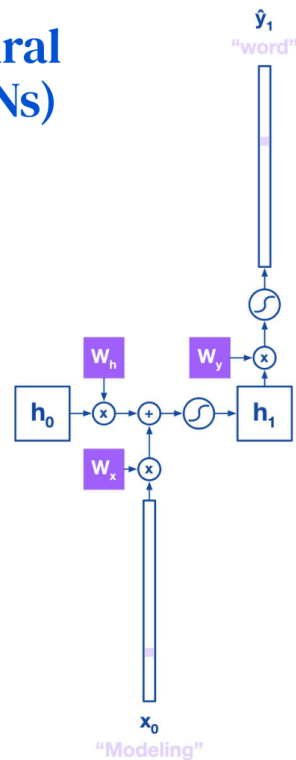


2.1 The RNN Recurrence

- State update: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$
- Prediction: $y_t = \text{softmax}(W_y h_t)$
- h_t is a running summary of $x_1, \dots, x_t$
- Same W_h, W_x, W_y shared over all t
- Unroll in time $\rightarrow$ a deep feedforward net

2.1 The RNN Recurrence

Recurrent Neural Networks (RNNs)



RNNs predict the target y (the next word) from the state h .

$$p(y_{t+1}) = \text{softmax}(W_y h_t)$$

Softmax ensures we obtain a distribution over all possible words.

Elman (1991)



2.2 Backprop Through Time

- Unroll the recurrence, then apply backprop
- Gradient at step t sums contributions from all later steps
- Involves products of Jacobians $\prod \frac{\partial h_k}{\partial h_{k-1}}$
- Truncate BPTT for very long sequences
- Cost grows with sequence length; steps are sequential

2.3 Vanishing Gradients & LSTM/GRU

- Repeated Jacobian products $\rightarrow$ gradients vanish/explode
- Plain RNNs forget long-range context
- LSTM: a **cell state** with additive updates + gates
- Gates (forget/input/output) control information flow
- GRU: a lighter gated variant; clip to tame explosions

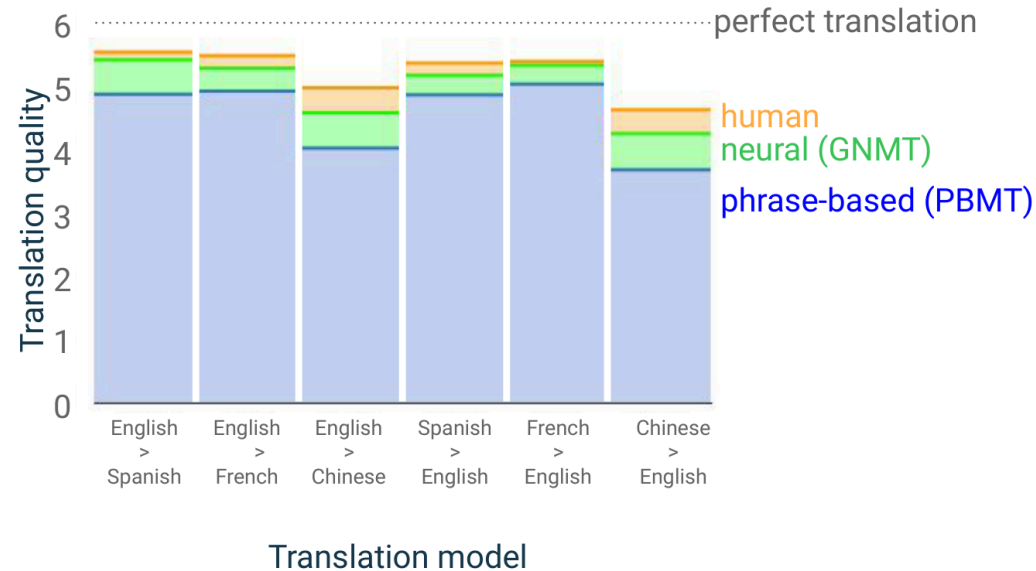
3 · Sequence to Sequence

3.1 Encoder-Decoder

- Encoder RNN reads the source into a context vector
- Decoder RNN generates the target, token by token
- Enabled end-to-end neural machine translation
- Closed much of the gap to human-quality translation
- One architecture for translation, summarization, dialogue

3.1 Encoder-Decoder

Google Neural Machine Translation



Closes gap between old system and human-quality translation by 58% to 87%.



3.2 The Bottleneck Problem

- All source information squeezed into **one** fixed vector
- Long sentences overflow the bottleneck → quality drops
- Decoder can't 'look back' at specific source words
- Fix: let the decoder attend to **all** encoder states
- This is attention — the bridge to Transformers

4 · Building Intuition for Attention

4.1 Tokens as a $D \times N$ Matrix

- Input: N tokens, each a D -dimensional vector
- Collect into $X^0 \in \mathbb{R}^{D \times N}$: features **down**, sequence **across**
- Words, image patches, amino-acid embeddings — all tokenise the same way
- Transformer returns X^M , another $D \times N$ representation
- Source: Turner, **An Introduction to Transformers** (arXiv:2304.10557)

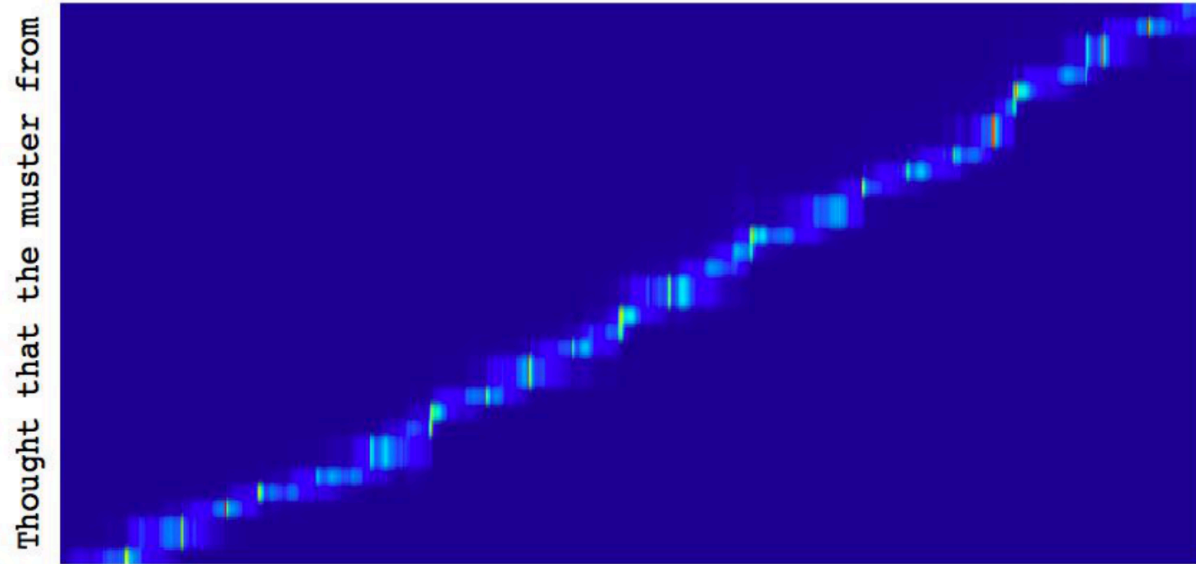
4.2 Attention = Weighted Linear Combination

- At position n , form a new vector by **averaging** all tokens
- $y_n = \sum_{j=1}^N x_j A_{jn}$ — a convex combination
- Weights $A_{jn} \geq 0$, columns sum to 1: $\sum_j A_{jn} = 1$
- High A_{jn} = position j is **relevant** for updating n
- Nothing exotic yet — just a learned weighted sum

4.3 Matrix Form: $Y = X A$

- Stack all positions: $Y = X A$ ($D \times N$)
- Each column of A is a softmax-normalised weight vector
- Stage 1 of the block: mix information **across the sequence**
- Operates row-wise on X (each feature independently)
- Stage 2 will refine features **down** each column

Alignment



I hought that the muster from



4.4 Self-Attention: Where Does A Come From?

- Naive: $A_{nj} \propto \exp(x_n^T x_j)$ — similarity of raw features
- Problem: similarity is entangled with content
- Fix: compare **projected** features Ux_n instead
- Still symmetric — ‘caulking iron’ $\leftrightarrow$ ‘tool’ but not vice versa
- Need **asymmetric** queries and keys (next slide)

4.5 Queries and Keys

- $q_n = U_q x_n, k_n = U_k x_n$ — two linear maps of the input
- $A_{nj} = \text{softmax}_j \exp(q_n^T k_j)$
- Only parameters so far: U_q, U_k ($K \times D$ each)
- Query = what position n is looking for
- Key = what position j offers; value = content (standard view)

Addressing by content

A **key vector** $\mathbf{k}$ is emitted by the controller and compared to the content of each memory location $\mathbf{M}[i]$ using a similarity measure $S(\cdot, \cdot)$ (e.g. **cosine distance**) then normalised with a **softmax**. A '**sharpness**' β is used to narrow the focus. Finds the memories '**closest**' to the key

$$\mathbf{w}[i] = \frac{\exp(\beta S(\mathbf{k}, \mathbf{M}[i]))}{\sum_j \exp(\beta S(\mathbf{k}, \mathbf{M}[j]))}$$



4.6 Multi-Head Self-Attention

- One $N \times N$ attention map can be a bottleneck
- Run H attention heads in parallel (different $U_{q,h}, U_{k,h}$)
- Each head learns a different notion of ‘relevance’
- Concatenate and project: $Y = \sum_h V_h X A_h$
- Analogous to multiple convolution filters (Turner)

4.7 Stage 2: MLP Across Features

- Second stage refines each token's feature vector independently
- $x_n = \text{MLP}(y_n)$ — same MLP weights at every position n
- Acts **vertically** down each column of X
- After M blocks, token n , feature d uses info from (j, k) anywhere
- Horizontal (attention) + vertical (MLP) = full mixing

4.8 Residual Connections and LayerNorm

- Parameterise updates as **residuals**: $x^m = x^{m-1} + \text{res}(x^{m-1})$
- Each stage applies a mild correction; depth composes large transforms
- LayerNorm standardises each feature dimension: zero mean, unit variance
- Prevents activations blowing up through deep stacks
- Full block: LN $\rightarrow$ MHSA $\rightarrow$ residual $\rightarrow$ LN $\rightarrow$ MLP $\rightarrow$ residual

4.9 Position Encoding and Causal Masking

- Attention alone is permutation-equivariant — order is lost
- Fix: add (or concatenate) a position embedding to each token
- Autoregressive training: mask A so $j > n$ gets weight 0
- Upper-triangular mask $\rightarrow$ train on full sequence in one pass
- Contrast RNNs: all time-points treated identically in attention

5 · Standard Notation and Why Transformers Won

5.1 Self-Attention in Translation

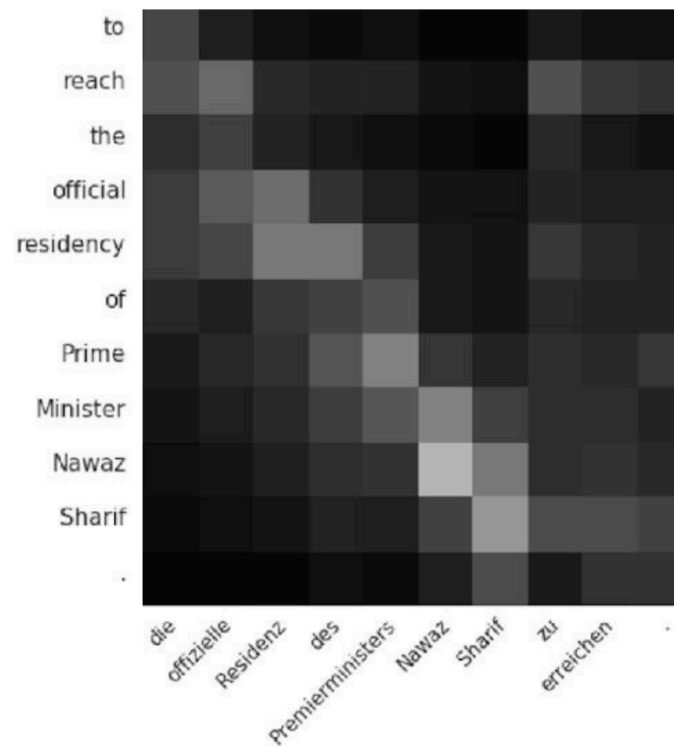
- Seq2seq attention = special case: decoder queries, encoder keys/values
- Self-attention: Q, K, V all from the same sequence
- Alignment matrix shows which source words each output used
- Interpretable, differentiable, no fixed bottleneck

5.1 Self-Attention in Translation

Implicit Attention
allows reordering
in machine
translation:

"to reach" -> "zu erreichen"

Neural Machine Translation in Linear Time,
Kalchbrenner et. al. (2016)



5.2 Scaled Dot-Product Attention (Batch Form)

- Standard ML notation: $X \in \mathbb{R}^{N \times d}$ (rows = tokens)
- $\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T \frac{1}{\sqrt{d_k}}\right)V$
- Same math as Turner; $\frac{1}{\sqrt{d_k}}$ keeps softmax gradients healthy
- Fully parallel over N — unlike sequential RNNs
- Cost: $O(N^2)$ in sequence length

5.3 Transformers vs RNNs

- RNN: nearby inputs treated differently from distant ones (recurrence)
- Transformer: every pair interacts in one layer, same treatment
- Constant path length → long-range deps no harder than short
- Parallel training → scales to internet-scale data (LLMs)
- Today = the **mechanism**; proteins/sequences: structural-bioinformatics course

5.4 Bridge to Week 2 (Days 9–10)

- We built the block; Day 9 turns it into a **production** decoder-only GPT
- Deferred to Day 9: tokenisation (BPE), RoPE, RMSNorm, GeGLU, GQA
- Deferred to Day 9: the full GPT forward pass + cross-entropy training loop
- Deferred to Day 9: nanoGPT walkthrough (build & train one fully)
- Deferred to Day 10: KV-cache inference, sampling, light post-training

- Day 5: **Sequence Models & Transformers**
- From recurrence to attention
- Questions welcome — practical follows

Questions?

Practical session follows — see course site.